Software Testing & Quality Assurance

Background Information for the Practical Sessions

YZ304/BS450 GitHub Repository

The materials for this module will be pushed to a GitHub repository:

https://github.com/omursahin/yz304-bs450

You will need to do a “pull” each week to obtain the latest content.

Java

All the code examples are in Java, and the tests are in JUnit.

To use the Java examples in the repository, you will need to have at **Java 11 or better** installed on your machine.

Maven

Java code examples are in a **Maven** library.

Once you have cloned the repository, you can compile and run tests at the terminal from the `apps` directory.

These commands should work on Mac/Linux/WSL/Windows.

```
mvn clean install
mvn test
mvn test -Dtest=uk.ac.shef.com3529.TriangleTest
mvn test -Dtest=uk.ac.shef.com3529.TriangleTest.shouldClassifyEquilateral
```

compile all code

run all tests

run all tests in a specific class

run a specific test

See the Maven website and documentation for more information:
https://maven.apache.org/

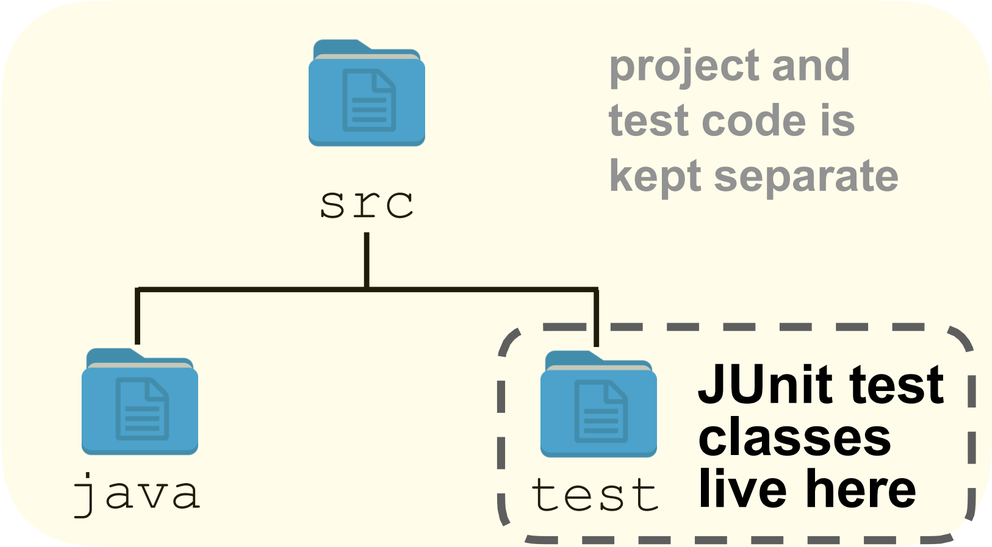
Use of Integrated Developer Environments (IDEs)

Most modern IDEs support Gradle, e.g. **IntelliJ IDEA** (recommended)

Just create a new project in the code directory and it should find the Gradle configuration.

From here, code will compile automatically and you can run specific tests through the IDE.

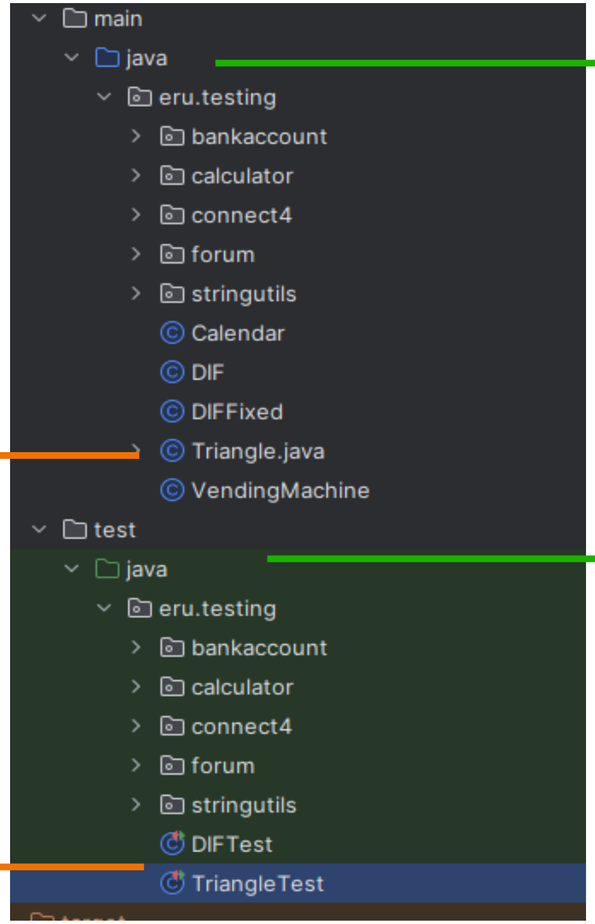
JUnit



Throughout this module, we'll be using JUnit 5

More here: https://junit.org/junit5/docs/current/user-guide/

Let's Test!



Production code goes in `src/main/java`

Test code goes in `src/test/java`

We're going to test the `Triangle.java` class with a JUnit test class called `TriangleTest`

A JUnit Test Class and a Test

```java
import org.junit.jupiter.api.Test;

public class TriangleTest {

    @Test
    public void shouldClassifyEquilateral() {
      // Test code goes here...
    }

    // ...
```

Tests are annotated with `@Test`
JUnit then knows which methods are test methods and which are helper methods

The Ingredients of a Test

```java
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.assertEquals;

public class TriangleTest {

    @Test
    public void shouldClassifyEquilateral() {
        Triangle.Type result = Triangle.classify(10, 10, 10);
        assertEquals(Triangle.Type.EQUILATERAL, result);
    }

    // ...
```

We start by making method call(s) to set up the test and to the part of the system we want to test.

The Ingredients of a Test

```java
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.assertEquals;

public class TriangleTest {

    @Test
    public void shouldClassifyEquilateral() {
        Triangle.Type result = Triangle.classify(10, 10, 10);
        assertEquals(Triangle.Type.EQUILATERAL, result);
    }

    // ...
```

We then write assertion statements to check the **actual result** is the one we **expected**.

JUnit Assertions

```java
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.assertEquals;

public class TriangleTest {

    @Test
    public void shouldClassifyEquilateral() {
        Triangle.Type result = Triangle.classify(10, 10, 10);
        assertEquals(Triangle.Type.EQUILATERAL, result);
    }

    // ...
```

The `assertEquals` method is a part of JUnit and specifically checks that some **expected value** is **equal** to the **actual one** returned from the unit being tested.

JUnit has a plethora of assertion types for checking relationships between actual and expected outputs.

These include `assertTrue(booleanVariable)`, `assertNull(reference)`, assertions on arrays and more. See:

https://junit.org/junit5/docs/current/api/org.junit.jupiter.api/org/junit/jupiter/api/Assertions.html

Checking for Exceptions with assertThrows

```java
@Test
public void shouldThrowExceptionWithInvalidTriangle() {
    assertThrows(InvalidTriangleException.class, () -> {
        Triangle.classify(0, 0, 0);
    });
}
```

```java
@Test
public void shouldThrowExceptionWithInvalidTriangle() {
    Exception e = assertThrows(InvalidTriangleException.class, () -> {
        Triangle.classify(0, 0, 0);
    });
    assertEquals("(0, 0, 0) is not a valid triangle", e.getMessage());
}
```

A more elaborate version that also checks the exception message.

Arguably such checks make the test more **brittle**.

"`import static`" means importing a static method from another class and using it as if it were in the current class

Each test method is annotated with `@Test`

Assert that a method's return value is as expected with `assertEquals`

Assert that an exception is thrown as expected with `assertThrows`

```java
package uk.ac.shef.com3529;

import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertThrows;

public class TriangleTest {

    @Test
    public void shouldClassifyEquilateral() {
        Triangle.Type result = Triangle.classify(10, 10, 10);
        assertEquals(Triangle.Type.EQUILATERAL, result);
    }

    @Test
    public void shouldClassifyIsoceles() {
        Triangle.Type result = Triangle.classify(5, 10, 10);
        assertEquals(Triangle.Type.ISOSCELES, result);
    }

    @Test
    public void shouldClassifyIsocelesWhenSidesAreOutOfOrder() {
        Triangle.Type result = Triangle.classify(10, 10, 5);
        assertEquals(Triangle.Type.ISOSCELES, result);
    }

    @Test
    public void shouldThrowExceptionWithInvalidTriangle() {
        assertThrows(InvalidTriangleException.class, () -> {
            Triangle.classify(0, 0, 0);
        });
    }
}
```